

Memoria del Projecte

Despertador amb Radio FM Real

ESP32-S3 - TEA5767 - DS3231 - ILI9341

Eduard Bravo Sanchez | Processadors Digitals | UPC 2025/2026

Projecte	Despertador amb Radio FM Real
Alumne	Eduard Bravo Sanchez
Assignatura	Processadors Digitals amb ESP32 - UPC 2025/2026
MCU	ESP32-S3 DevKitC-1
Document	Memoria final del projecte
Data	20/06/2026

Index

1. Introduccio i Objectius
2. Arquitectura de Hardware
3. Arquitectura de Software
4. Interficie i Manual d'Usuari
5. Problemes Trobats i Solucions
6. Conclusions i Millores Futures

1. Introduccio i Objectius

Descripcio general

Aquest projecte consisteix en el disseny i la implementacio d'un **despertador digital autonom amb sintonitzador de radio FM real**, basat en el microcontrolador ESP32-S3. A diferencia d'un despertador convencional amb un simple buzzer, el dispositiu permet programar una alarma que es desperta amb una emissora de radio FM captada per antena, de la mateixa manera que ho faria una radio domestica tradicional. El sistema integra un rellotge en temps real amb bateria de backup, una pantalla TFT a color per mostrar tota la informacio rellevant, i cinc polsadors que permeten configurar-lo completament sense necessitat de cap aplicacio mobil ni connexio a internet.

Motivacio

La motivacio principal d'aquest projecte ha estat aplicar de forma practica i integrada els conceptes treballats al llarg de l'assignatura de Processadors Digitals: la gestio de multiples periferics simultanis, l'us de sistemes operatius en temps real per coordinar tasques concurrents, i la integracio de diferents buses de comunicacio (I2C i SPI) en un unic projecte coherent. Un despertador amb radio FM és especialment interessant porque obliga a resoldre un problema real de concurrencia: cal llegir l'hora, comprovar l'alarma, gestionar el sintonitzador i actualitzar la pantalla, tot això sense que cap d'aquestes tasques bloquegi les altres ni provoqui retards perceptibles per l'usuari.

Objectius complets

- Implementacio d'un sistema multitasca real amb **FreeRTOS**, amb tres tasques independents que es comuniquen mitjançant mecanismes de sincronitzacio segurs (mutex, semafor i cua).
- Gestio simultania de dos periferics sobre el mateix bus **I2C** (TEA5767 i DS3231), incloent la proteccio de l'acces concurrent al bus.
- Control d'un display **SPI** (TFT ILI9341) amb actualitzacio parcial de pantalla per evitar parpelleig i reduir la carrega de CPU.
- Programacio a baix nivell del registre I2C del TEA5767, implementant manualment la formula PLL del datasheet per calcular i interpretar frequencies de radio.
- Persistencia de dades a memoria flash (NVS) porque la configuracio de l'usuari (alarma, frecuencia, mode mono/stereo) es conservi entre reinicis.
- Disseny complet de l'esquema electric del circuit amb KiCad 10.0.

2. Arquitectura de Hardware

Llistat de components

Component	Funcio	Detalls
ESP32-S3 DevKitC-1	Microcontrolador principal	WiFi/BLE 802.11 b/g/n, 32 bits, dual core
TEA5767	Sintonitzador FM	Bus I2C, adreca 0x60, sortida audio analogica directa
DS3231	Rellotge en temps real (RTC)	Bus I2C, adreca 0x68, bateria CR2032 de backup
ILI9341	Pantalla TFT 2.4" 320x240	Bus SPI, no tactil
5x Polsador 12mm	Interficie d'usuari	FM+, FM-, SET, SNOOZE, MONO/ST

Buses de comunicacio

El sistema utilitza dos buses de comunicacio diferenciats segons les necessitats de cada perifèric:

- **Bus I2C** (pins GPIO8=SDA, GPIO9=SCL): compartit entre el TEA5767 i el DS3231. Tots dos dispositius es connecten en paral·lel a les mateixes línies SDA i SCL, distingint-se per la seva adreça I2C propia (0x60 i 0x68 respectivament). Aquest bus es prou rapid per a la configuracio dels dos chips, pero no per a la transferencia de dades d'alta velocitat com les que necessita una pantalla.
- **Bus SPI** (pins GPIO10=CS, GPIO11=MOSI, GPIO12=SCK, GPIO13=DC, GPIO14=RESET): dedicat exclusivament a la pantalla TFT ILI9341. SPI es necessari aqui perque I2C (limitat a uns 400 kHz) seria massa lent per refrescar una pantalla a color de 320x240 pixels amb fluïdesa; SPI permet velocitats de fins a 40 MHz.

Esquema electric

L'esquema electric complet del circuit s'ha dissenyat amb **KiCad 10.0** i es troba adjunt en el fitxer esquema_electrico.pdf, generat a partir del fitxer font fgdgegrrgg.kicad_sch. A continuacio es mostra una captura de l'esquema:

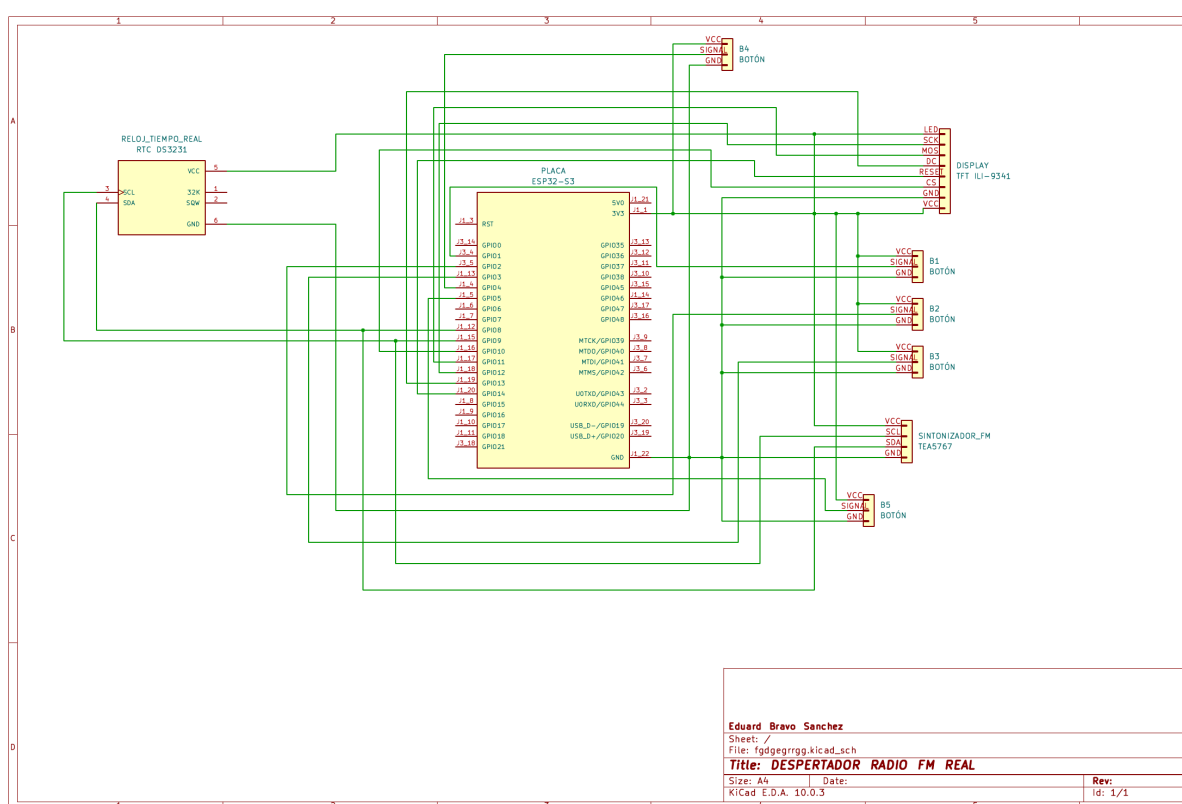


Figura 1. Esquema electric complet del despertador (KiCad 10.0).

Cal destacar que, per simplificar la representacio dels moduls comercials (breakout boards) del TEA5767 i del DS3231 a KiCad, aquests s'han modelat amb connectors generics amb les seves quatre línies (VCC, GND, SDA, SCL) en lloc de cercar un símbol exacte del fabricant, que sovint no existeix a les llibreries oficials. Aquesta solucio es discuteix amb mes detall a l'apartat 5.

3. Arquitectura de Software

Paradigma FreeRTOS

El firmware no es basa en un únic bucle `loop()` com es habitual en projectes Arduino senzills. En el seu lloc, s'utilitza **FreeRTOS** per dividir la logica en tres tasques independents que s'executen de forma aparentment simultania.

La rao principal per descartar el model `loop()` tradicional es que el sistema ha de fer tres coses a la vegada: llegir l'hora del RTC, comprovar si l'alarma ha de sonar, i atendre els polsadors mentre refresca la pantalla. En un bucle unic, qualsevol operacio lenta (per exemple, esperar la resposta del TEA5767 durant una cerca de frequencia, que pot trigar fins a 5 segons) bloquejaria completament la lectura dels botons i la comprovacio de l'alarma durant tot aquest temps. Amb FreeRTOS, cada tasca te la seva propia pila i prioritat, i el planificador (scheduler) reparteix el temps de CPU entre elles, garantint que una tasca lenta no en bloquegi una altra de critica.

Descripcio de les tasques

Tasca	Prioritat	Funcio
task_rtc	3	Llegeix el DS3231 cada segon i actualitza la variable global <code>g_now</code> . Tambe llegeix el bit STEREO del TEA5767 per saber si l'emissora actual emet en estereo.
task_alarm	4 (maxima)	Compara constantment l'hora actual amb l'hora programada de l'alarma. Quan coincideixen, desactiva el mute del TEA5767 perquè comenci a sonar i notifica a <code>task_ui</code> .
task_ui	2 (minima)	Llegeix els 5 polsadors, gestiona la maquina d'estats del menu de configuracio, i refresca la pantalla TFT cada segon o quan canvia algun valor rellevant.

Les prioritats no son arbitraries: `task_alarm` te la prioritat mes alta perquè el moment exacte en que es dispara l'alarma es critic per l'usuari. `task_rtc` te prioritat mitjana perquè necessita actualitzar-se amb regularitat pero no es tan critica com l'alarma. `task_ui` te la prioritat mes baixa perquè un petit retard en refrescar la pantalla (uns mil·lisegons) es totalment imperceptible per l'ull huma.

Mecanismes de sincronitzacio

Quan multiples tasques accedeixen a recursos compartits (el bus I2C, o variables globals com l'estat de l'alarma), cal protegir aquest acces per evitar condicions de carrera (race conditions) que podrien corrompre dades o bloquejar el sistema. S'han fet servir tres mecanismes diferents de FreeRTOS, cadascun pel proposit que millor s'hi adapta:

Mecanisme	Proposit	Justificacio
Mutex (<code>mutex_i2c</code>)	Protegir l'acces exclusiu al bus I2C	TEA5767 i DS3231 comparteixen fisicament les linies SDA/SCL. Si <code>task_rtc</code> i <code>task_alarm</code> intentessin escriure al bus simultaniament, la transaccio I2C de totes dues es corrompria. Cada acces es fa dins d'un bloc <code>xSemaphoreTake/xSemaphoreGive</code> amb timeout.
Semafor binari (<code>sem_alarm</code>)	Notificar que l'alarma s'ha disparat	Mecanisme lleuger de senyal entre <code>task_alarm</code> i la resta del sistema quan es produeix l'event de disparament.
Cua (<code>q_alarm_event</code>)	Enviar esdeveniments d'estat	<code>task_alarm</code> envia missatges (<code>EVT_FIRE</code> , <code>EVT_SNOOZE</code> , <code>EVT_OFF</code>) a traves de la cua perquè <code>task_ui</code> pugui reaccionar-hi sense necessitat de consultar variables compartides directament.

Un detall important del disseny es que, si una tasca no aconsegueix el mutex en el temps maxim establert (100-200 ms segons el cas), simplement descarta l'operacio en aquell cicle i ho reintenta mes tard, en lloc de quedar-se bloquejada indefinidament esperant. Això evita que un fallo puntual al bus I2C pugui congelar tot el sistema.

4. Interfície i Manual d'Usuari

Pantalla principal

La pantalla TFT es divideix en tres zones que es refresquen de forma independent per evitar parpelleig innecessari:

- **Hora actual** en format HH:MM en text gran, amb els segons en petit al costat per confirmar visualment que el rellotge esta funcionant.
- **Frequencia FM** sintonitzada actualment, en format XX.X MHz, juntament amb l'indicador de mode **MONO** o **STEREO** segons el que detecta el chip.
- **Estat de l'alarma**: mostra si esta activada i a quina hora, si esta en mode snooze, o si esta sonant en aquell moment (en aquest cas, en color vermell).

Mapeig dels polsadors

Boto	Mode normal	Mode alternatiu / ajust
FM+	Cercar emissora cap amunt (seek automatic)	Incrementar hora/minut (durant ajust)
FM-	Cercar emissora cap avall (seek automatic)	Decrementar hora/minut (durant ajust)
SET	Polsacio curta: entrar/avancar en ajust d'alarma	Polsacio llarga (3s): ajustar hora del rellotge sense ordinador
SNOOZE	Sense alarma activa: cap accio prevista (vegeu millores)	Amb alarma sonant: 1r pols posposa 5 min, 2n pols apaga
MONO/ST	Alternar entre mode estereo automatic i mono forcat	—

El boto **SET** es el que dona acces al menu de configuracio complet: una primera polsacio curta entra en mode d'ajust de l'hora de l'alarma (primer hora, despres minut, confirmant cada pas amb una nova polsacio de SET), mentre que mantenir-lo premut durant tres segons des de la pantalla principal entra directament en el mode d'ajust del rellotge del sistema, pensat per poder configurar l'hora el dia de la presentacio sense necessitat d'ordinador.

5. Problemes Trobats i Solucions

Integracio de hardware: representacio a KiCad

Les llibreries oficials de KiCad inclouen el simbol del chip **ESP32-S3-WROOM-1**, pero no existeixen simbols especifics per als moduls breakout comercials del TEA5767 o del DS3231 que s'utilitzen habitualment en projectes d'aficionat. Cercar i fabricar un simbol exacte per a cada modul hauria suposat una inversio de temps desproporcionada respecte al benefici real. La solucio adoptada ha estat representar aquests moduls com a **connectors generics** amb les seves quatre linies essencials (VCC, GND, SDA, SCL), etiquetades clarament amb el nom del modul real. Aquest enfocament es habitual en projectes amb components comercials i no afecta la validesa de l'esquema, ja que el que importa electricament son les connexions, no la forma grafica del simbol.

Desenvolupament de software: conflictes al bus I2C

Durant les primeres proves, en tenir el TEA5767 i el DS3231 al mateix bus sense cap proteccio, es van detectar lectures errones intermitents del rellotge quan coincidien temporalment amb una operacio de cerca de frecuencia FM. El problema es va resoldre introduint el `mutex_i2c`, que obliga qualsevol tasca a obtenir permis exclusiu abans d'accedir al bus, eliminant completament les col·lisions.

Prioritats de les tasques FreeRTOS

Inicialment totes les tasques es van crear amb la mateixa prioritat, la qual cosa provocava que, en moments de carrega alta (per exemple, durant una cerca de freqüència que manté el bus I2C ocupat fins a 5 segons), la lectura dels polsadors es notes amb retard. Ajustant les prioritats segons la criticitat real de cada tasca (alarma > rellotge > interfície), aquest problema es va resoldre sense necessitat de canviar la lògica interna de cap tasca.

Calcul de freqüència del TEA5767

El seek automatic del TEA5767 utilitzant només les funcions d'alt nivell de la llibreria resultava poc fiable: en ocasions retornava la mateixa freqüència que ja s'estava escoltant. Es va solucionar escrivint manualment els registres I2C del chip segons el datasheet, aplicant la fórmula del PLL ($PLL = 4 * (freq_kHz + 225) / 32.768$), i afegint una lògica de reintents que garanteix que el resultat final sigui sempre una freqüència diferent a la de partida.

6. Conclusions i Millores Futures

Valoracio personal

El desenvolupament d'aquest projecte ha permetes consolidar de manera molt practica conceptes que, en teoria, poden semblar abstractes: la diferencia real entre programar amb un `loop()` i programar amb tasques concurrents nomes es fa evident quan s'ha d'evitar que una operacio lenta en bloquegi una altra de critica. Tambe ha estat especialment formatiu haver de baixar al nivell de registre del TEA5767, llegint directament el datasheet i implementant la formula del PLL a ma, en lloc de confiar cegament en una llibreria d'alt nivell que no sempre es comportava com s'esperava. Això ha ajudat a entendre molt millor com funciona realment la comunicacio I2C per sota de les abstraccions del framework Arduino.

Propostes de futures versions

- **Sincronitzacio horaria per NTP:** aprofitant que l'ESP32-S3 incorpora WiFi, el rellotge es podria sincronitzar automaticament amb un servidor de temps per internet, eliminant la necessitat d'ajustar manualment l'hora i corregint qualsevol deriva del RTC amb el temps.
- **Emissores favorites a memoria flash:** guardar una llista de presets de frequencies FM preferides a NVS, accessibles amb una combinacio de polsadors, evitant haver de fer un seek complet cada vegada que es vol tornar a una emissora coneguda.
- **Reaprofitament del boto SNOOZE com a mute:** actualment aquest boto nomes te efecte quan l'alarma esta sonant; una millora natural seria utilitzar-lo tambe en mode normal per silenciar o activar la radio sota demanda, evitant que soni continuament mentre el dispositiu esta alimentat.
- **Integracio amb MQTT / Home Assistant:** publicar l'estat del despertador (alarma, frecuencia, hora) a un broker MQTT permetria controlar-lo des d'un panell domotic o fins i tot des d'un agent d'intel·ligencia artificial.
- **Migracio a interrupcions GPIO:** substituir la lectura per polling dels polsadors per interrupcions hardware reduiria la carrega de CPU de `task_u1` i milloraria l'eficiencia energetica del sistema.

Memoria del projecte Despertador amb Radio FM Real. Eduard Bravo Sanchez | Processadors Digitals | UPC 2025/2026.